



Получите новый уровень в карьере

Повышайте свою квалификацию



Хабр



Чему научиться в этом году?



hahaton

12 часов назад

Labeled break and continue в C# 15 — разбор плохого примера и поиск реального кейса

🔒 Средний 🕒 4 мин 👁 6.4K

.NET*, C#*, Программирование*, Качество кода*, Проектирование и рефакторинг*

Мнение

Из песочницы

Всем привет. В последнее время в одной профессиональной соцсети я все чаще стал наткаться на посты, связанные с dotnet C# тематикой. К сожалению, эти посты в большинстве своем не содержат полезной информации. Скорее всего они создаются для охвата аудитории с целью привлечения трафика на сторонние платформы по продаже курсов для разработчиков. По-моему, этот способ называется "воронка продаж" (поправьте, если я ошибаюсь). Как правило, эти посты затрагивают какую-то не очень сложную тему и содержат примеры кода. Недавно мне попался очередной пост, в котором автор пытался показать пример использования новой фичи `labeled break and continue`. Это новая фича, которую добавили в C# 15 (dotnet 11). На момент написания она была принята в Working Set, но финального релиза ещё не было. Ниже код, похожий на оригинал из поста. Он разделен на 2 секции: "как делали раньше" и "как сделать используя новый подход":

Стандартный способ:

```
string foundValue = null;
bool shouldBreak = false;

for (int x = 0; x < xMax; x++)
{
    for (int y = 0; y < yMax; y++)
    {
        foundValue = GetValue(x, y);

        if (foundValue == targetValue)
        {
            shouldBreak = true;
            break;
        }
    }

    if (shouldBreak)
    {
        break;
    }
}

ProcessValue(foundValue);
```

Новый способ:

```
outer: for (int x = 0; x < xMax; x++)
{
    for (int y = 0; y < yMax; y++)
    {
        if (ShouldSkipRest(x, y))
        {
            continue outer;
        }

        if (ShouldExitAll(x, y))
        {
            break outer;
        }
    }
}
```

Объяснить с 

Выскажу сугубо личное мнение: оба варианта написаны плохо. Разработчики программного обеспечения должны стремиться писать читаемый код. Так же нужно заботиться об удобстве его отладки, но это уже вторично. В 2017 года "Sonar Source" предложила метрику `Cognitive Complexity`, которая помогает измерить непреднамеренную сложность написанного кода. Я часто использую ее. У меня установлены соответствующие расширения во всех IDE. Теперь вернемся к примерам из поста.

В первом примере мы видим большую вложенность. Это и двойной цикл `for` и вложенный оператор выбора `if`. Конечно, можно заметить, что в целом кода немного. Но в проектах такое встречается не всегда. И каждый из циклов `for` может содержать дополнительную бизнес-логику. В этом случае чтение и понимание кода усложняется.

Вот как бы я реализовал этот код:

```
public void Process()
{
    string? foundValue = null;

    for (int x = 0; x < xMax; x++)
    {
        if (TryGetValue(x, out foundValue))
        {
            ProcessValue(foundValue);

            return;
        }
    }
}

private bool TryGetValue(int x, out string? outputValue)
{
    {
```

```

        outputValue = null;

        for (int y = 0; y < yMax; y++)
        {
            var currentValue = GetValue(x, y);

            if (currentValue == targetValue) // targetValue is a class-level constant
            {
                outputValue = currentValue;

                return true;
            }
        }

        return false;
    }

```

Объяснить с 

Такой код уже лучше читается. В явном виде отсутствуют вложенные циклы `for`. Удобнее дебажить код для внешнего цикла `for` по переменной `x`. Так же можно уменьшить "Cognitive Complexity" если инвертировать оператор выбора `if (TryGetValue(x, out foundValue))`. Часто IDE предлагает такой вариант рефакторинга.

Второй пример выглядит более компактно, хотя имеет такую же степень вложенности. Но куда хуже то, что он содержит два вложенных оператора выбора `if`, которые задают разное поведение и относятся не к ближайшему циклу `for`. В общем, читать такой код сложно. Я считаю, что проще было бы поместить его в отдельный метод и добавить конструкцию `return` там, где это необходимо.

Помимо проблем с читаемостью, первый пример содержит скрытый баг:

`ProcessValue(foundValue)` вызывается безусловно, даже если значение так и не было найдено. Но ещё важнее другое: оба примера решают **разные задачи** - первый ищет конкретное значение, второй использует абстрактные методы-заглушки. Это не "старый и новый способ" - это два несвязанных куска кода, и сравнивать их как эквиваленты вообще некорректно.

В общем у меня сложилось впечатление, что фича `labeled break and continue` имеет сомнительную пользу для разработчиков. В то же время я подумал, что люди, проектирующие C#, далеко не глупые и вряд ли добавили бы сомнительную фичу. И я решил поискать историю появления этой фичи. Оказалось, что сообщество запрашивало ее довольно давно. Один из первых запросов датируется октябрём 2015 года [First mention](#). Вот еще пример обсуждения из 2018 года [GitHub discussion](#). Однако, ни одна из дискуссий не привела к разработке этой фичи. Основная причина отказа заключалась в том, что среди членов LDM (Language Design Meeting) не было человека, который бы взял на себя ответственность за разработку. По-моему, такие люди называются "champion". Однако, все изменилось 14 декабря 2025 года. Предложение было оформлено и внесено в официальный трекер **Cyrus Najmabadi** - членом команды C# Language Design. Ссылка [Official feature link](#). На данный момент фича все еще в статусе `Open`. Посмотрим, к чему в итоге приведет ее разработка и в каком виде она предстанет перед нами в релизе.

Однако, все еще остается вопрос - зачем это нужно? Вероятнее всего, наиболее подходящий кейс для этой фичи - это ситуация, когда внутри цикла находится switch-оператор, и из его `case` нужно выйти не из `switch`, а из самого цикла. Без `labeled break and continue` это невозможно сделать без `goto` или других костылей вроде вложенных функций или возвращаемых `tuple`. Пример такого кода может выглядеть следующим образом:

```
outer: foreach (var item in collection)
{
    switch (item.Type)
    {
        case ItemType.StopAll:
            break outer; // без labeled break нужно вызывать goto
        case ItemType.SkipGroup:
            continue outer; // без labeled break нужно вызывать goto
    }
}
```

[Объяснить с](#) 

Более подробную информацию о всех возможных кейсах можно посмотреть на официальной странице [Use cases](#).

Спасибо всем, кто дочитал до конца мой пост. Буду благодарен за обратную связь.

Теги: [C# 15](#), [labeled break](#), [continue](#), [вложенные циклы](#), [switch](#), [goto](#), [cognitive complexity](#), [рефакторинг](#)

Хэбы: [.NET](#), [C#](#), [Программирование](#), [Качество кода](#), [Проектирование](#) и [рефакторинг](#)



0



4



1

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



4K+

1

Охват за 30 дней

Карма

[@hahaton](#)

Пользователь

Подписаться



Поток Бэкенд доступен 24/7 благодаря поддержке друзей Хабра

Хабр Курсы для бэкендеров

РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

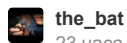
[Перейти в поток Бэкенд](#)

 Комментарии 1

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники



interpres

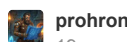
18 часов назад

Электроинструмент становится хуже, и это делается намеренно



Обзор

Перевод



prohronus

19 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере



Кейс



alizar

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц



Обзор



TrexSelectel

19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки



Обзор



AlexSerbul

23 часа назад

Программирование с AI-ассистентом — похороните меня под плинтусом



Мнение

+26

33

11



mariklolik

18 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

Средний

7 мин

7.4K

Кейс

+23

23

6



nick_dyuba

22 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TіrīTіr

5 мин

7.1K

Recovery Mode

+22

8

5



Mimizavr

23 часа назад

«Великое очищение» в работе с контентом: что осталось от роли редактора

11 мин

6.9K

+17

9

7



infgotoinf

10 часов назад

Perl — зря забытый язык программирования?

11 мин

8.4K

Из песочницы

+16

13

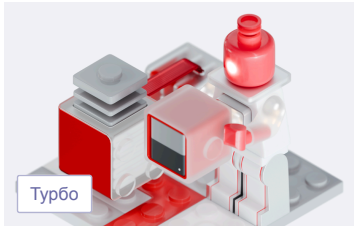
8

Тысяча нюансов: как управлять производством лекарств

Турбо

Показать еще

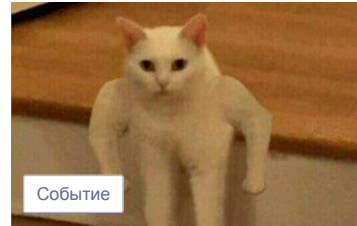
МИНУТОЧКУ ВНИМАНИЯ



Собери облачный пазл и выиграй призы



Удалёнка в фарме: большие данные и никакой магии

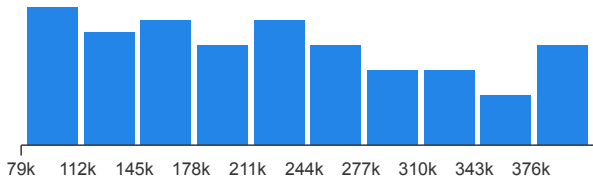


Качаем к лету хард- и софт-скиллы на айтишных ивентах

СРЕДНЯЯ ЗАРПЛАТА В IT

221 623 ₽/мес.

— средняя зарплата во всех IT-специализациях по данным из 32 194 анкет, за 1-ое пол. 2026 года. Проверьте «в рынке» ли ваша зарплата или нет!



[Проверить свою зарплату](#)

ЧИТАЮТ СЕЙЧАС

Электроинструмент становится хуже, и это делается намеренно

 26K  82

Тим Кук покидает пост Apple. Новый глава — «отец» Apple Silicon Джон Тернус

 13K  14

Представлен проект KillerPDF — редактор PDF с открытым исходным кодом для Windows 10/11

 6.9K  15

Пользователь купил Atomic Heart на VK Play, но не смог запустить игру из-за сетевых проблем

 5.4K  11

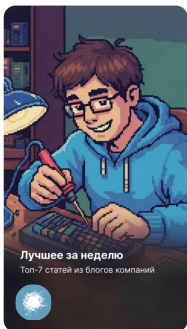
+185% за 13 часов: как Kimi K2.6 переписала 8-летний движок

 14K  10

Тысяча нюансов: как управлять производством лекарств

Турбо

ИСТОРИИ



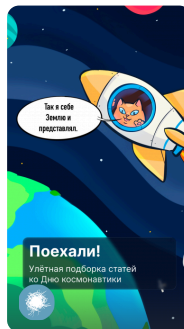
Годнота из блогов компаний



Только 10% решат эту головоломку



Хочу 450K



Вперёд к звёздам



Там на неведомых дорожках...



Самое время поспать

Ваш аккаунт

Войти

Регистрация

Разделы

Статьи

Новости

Хабы

Компании

Авторы

Песочница

Информация

Устройство сайта

Для авторов

Для компаний

Документы

Соглашение

Конфиденциальность

Услуги

Корпоративный блог

Медийная реклама

Нативные проекты

Образовательные программы

Стартапам



Настройка языка

Техническая поддержка

© 2006–2026, Habr